

Các Phương pháp Tối ưu hóa Cache

[Đỗ Hữu Khang, Lê Đình Ngọc Thành, 3, 4]

University of Transport HoChiMinh City

1 Giới thiệu

Nhằm vượt qua "bức tường bộ nhớ"(memory wall) giữa CPU và DRAM, tối ưu hóa hệ thống cache là bắt buộc. Bài viết phân tích bốn giải pháp tối ưu hóa trải dài từ quản lý bộ nhớ ảo cấp hệ điều hành, cấu trúc phần cứng đa tầng, cho đến các cơ chế tiên nập dự đoán và

2 Phần 1: Bộ nhớ ảo và Mở rộng Bộ nhớ bằng SSDAlloc

2.1 Khái niệm Bộ nhớ ảo và Hạn chế của Swap Truyền thống

Bộ nhớ ảo mở rộng không gian địa chỉ vượt quá RAM vật lý bằng cách ánh xạ địa chỉ ảo sang địa chỉ vật lý thông qua bảng trang (Page Table). Tuy nhiên, cơ chế Swap truyền thống của hệ điều hành hoạt động ở mức độ trang cứng nhắc (page granularity - 4KB). Khi xảy ra lỗi trang (Page Fault), OS buộc phải nạp toàn bộ trang 4KB từ SSD vào DRAM ngay cả khi ứng dụng chỉ cần truy cập một đối tượng vài chục byte, gây lãng phí dung lượng DRAM vật lý do nạp kèm các đối tượng "lạnh" (ít sử dụng). Hơn nữa, khi một đối tượng bị thay đổi, OS phải ghi lại cả trang 4KB xuống SSD, dẫn đến hiện tượng khuếch đại ghi (Write Amplification) nghiêm trọng, làm giảm nhanh tuổi thọ và hiệu năng của SSD.

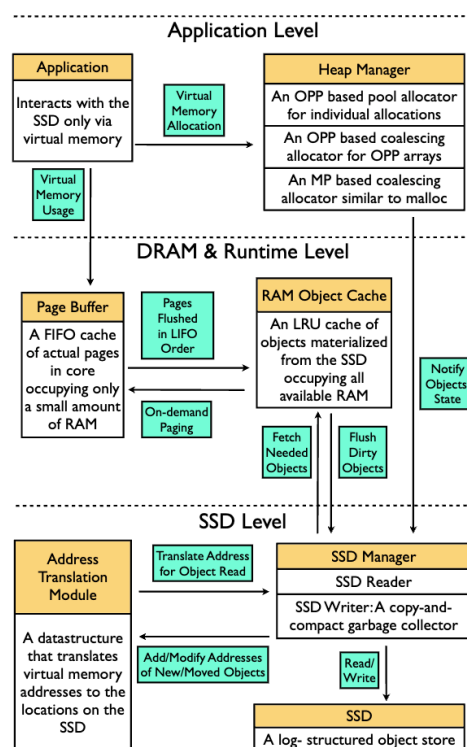
2.2 Kiến trúc và Cơ chế Hoạt động của SSDAlloc so với Swap Truyền thống

Nghiên cứu từ Đại học Princeton đề xuất **SSDAlloc** [1] - trình quản lý bộ nhớ lai DRAM/SSD hoạt động ở mức độ đối tượng (object granularity) nhằm tối ưu hóa hiệu năng và độ bền của bộ nhớ flash. Bằng cách định nghĩa không gian địa chỉ riêng cho từng đối tượng, SSDAlloc thay đổi toàn diện cách thức tương tác với bộ nhớ ảo:

- **Đơn vị quản lý (Granularity):** Swap truyền thống chuyển đổi dữ liệu ở mức trang 4KB thô. SSDAlloc quản lý bộ nhớ ở mức đối tượng (object granularity), cho phép hệ thống chỉ nạp hoặc giải phóng chính xác đối tượng được ứng dụng yêu cầu, giải phóng tối đa băng thông I/O và tối ưu hóa DRAM.
- **Đánh chặn lỗi trang (Page Fault Interception):** Thay vì dựa hoàn toàn vào MMU phần cứng và Kernel để xử lý lỗi trang, SSDAlloc áp dụng mô hình **OPP (One-Page-per-Object)** và lệnh hệ thống mprotect để khóa các trang ảo của đối tượng đã giải phóng xuống SSD. Truy cập bất hợp lệ sẽ kích hoạt tín hiệu SIGSEGV được xử lý ngay tại User

Space bởi thư viện SSDAlloc, dịch địa chỉ nhanh qua cây ATM (Address Translation Map) dạng nhị phân cân bằng thay vì duyệt Page Table của OS.

- **Bộ đệm lai (Hybrid Buffer):** Khác với Swap thông thường chỉ có một bộ đệm trang hệ thống, SSDAlloc chia DRAM thành hai vùng chuyên biệt: *Page Buffer* (cửa sổ trượt chứa các trang thực tế phục vụ CPU xử lý trực tiếp) và *RAM Object Cache* (chứa các đối tượng nén dạng LRU nhằm tối đa hóa số lượng đối tượng lưu trữ trong DRAM vật lý).
- **Quản lý Ghi dữ liệu (Write Management):** Thay vì ghi ngẫu nhiên từng trang 4KB gây tổn hại SSD, SSDAlloc tổ chức ghi dạng log-structured tuần tự. Hệ thống gom cụm (batching) các đối tượng nhỏ dưới 128 byte và căn hàng sector 512 byte trước khi ghi xuống đĩa, triệt tiêu đáng kể hiện tượng Write Amplification.
- **Dọn rác (Garbage Collector):** Thay vì giải phóng trang ngẫu nhiên của Swap thông thường, SSDAlloc sử dụng cơ chế *Copy-and-Compact* tuần tự trên SSD. Việc kiểm tra tính sống sót (liveness) của đối tượng được thực hiện tức thì nhờ tra cứu cấu trúc con trỏ ngược (OTID, OTO) trong Object Table được lưu trực tiếp tại DRAM.



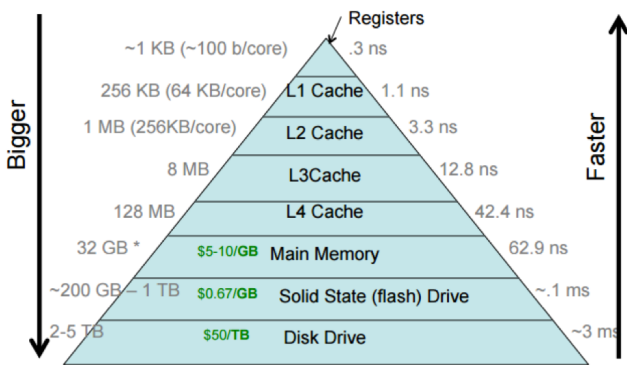
Hình 1: Kiến trúc tổng quan ba tầng của hệ thống SSDAlloc.

3 Phần 2: Cache Đa cấp (Multilevel Cache)

3.1 Cấu trúc Phân cấp Bộ nhớ đệm

Trong kiến trúc hiện đại, bộ nhớ đệm được tổ chức thành hệ thống phân cấp nhằm cân bằng giữa tốc độ và dung lượng:

- **L1 Cache:** Nằm trực tiếp trong nhân CPU, dung lượng nhỏ (32-128 KB) nhưng tốc độ cực nhanh.
- **L2 Cache:** Đóng vai trò đệm cho L1, dung lượng lớn hơn (đến 2 MB).
- **L3 Cache:** Thường được chia sẻ chung giữa tất cả các nhân xử lý, giữ vai trò "chốt chặn" trước khi dữ liệu phải truy xuất từ RAM.



Hình 2: Sơ đồ phân cấp bộ nhớ trong hệ thống máy tính.

3.2 Cơ chế Ánh xạ và Quản lý dữ liệu

- **Address Mapping (Ánh xạ địa chỉ):** Sử dụng các trường logic như *Index*, *Tag*, *Offset* để định vị nhanh khối dữ liệu.
- **Nguyên lý Cục bộ (Principle of Locality):** Cache giữ lại dữ liệu dựa trên cục bộ thời gian (truy cập lại dữ liệu vừa dùng) và cục bộ không gian (truy cập các dữ liệu lân cận). Thuật toán phổ biến là LRU.
- **Chính sách ghi:** Kỹ thuật *Write-back* được sử dụng phổ biến để giảm lượt ghi không cần thiết xuống RAM.

3.3 Che giấu độ trễ và Ứng dụng Phân tán

Latency Hiding: Hệ thống Cache đa tầng giúp che giấu độ trễ truy cập RAM, kéo giảm thời gian truy cập trung bình (AMAT) và hạn chế CPU bị rơi vào trạng thái chờ (stall).

Ứng dụng Phân tán: Ở quy mô mạng (WAN), Caching được hiện thực thông qua các mạng phân phối nội dung (CDN như Cloudflare, Akamai). Máy chủ Edge đóng vai trò làm Cache, đưa nội dung đến gần người dùng nhất nhằm triệt tiêu độ trễ truyền tải.

Distribute-Caching/cache_notation.png

Hình 3: Các ký hiệu dùng trong mô hình toán học đánh giá hiệu năng Cache đa tầng.

Dựa trên các tham số học thuật đó, công thức Thời gian truy cập trung bình (AMAT - Average Memory Access Time) đối với kiến trúc Cache 2 tầng được tính bằng:

$$AMAT = T_1 + MR_1 \times (T_2 + MR_2 \times T_{\text{memory}})$$

Trong đó T_1, T_2 là thời gian truy cập (Hit Time) và MR_1, MR_2 là tỷ lệ trượt (Miss Rate) tương ứng của Cache L1 và L2. Tầng Cache càng sâu, Miss Penalty (hình phạt trượt) càng giảm, giúp triệt tiêu trạng thái chờ (stall) của CPU.

4 Phần 3: Nạp trước dữ liệu (Pre-caching)

4.1 Nguyên lý của Precaching

Dự đoán các dữ liệu hoặc lệnh mà CPU sẽ cần trong tương lai gần và nạp chúng vào cache trước khi chúng thực sự được yêu cầu, giúp giảm thời gian chờ đợi dữ liệu từ bộ nhớ chính, từ đó tăng tốc hiệu suất tổng thể

4.2 Phân loại Kỹ thuật Prefetching

1. **Hardware Prefetching:** Việc nạp trước dữ liệu dựa trên phần cứng thường được thực hiện bằng cách sử dụng một cơ chế phần cứng chuyên dụng trong bộ xử lý để theo dõi luồng lệnh hoặc dữ liệu được chương trình đang thực thi yêu cầu, nhận biết một vài phần tử tiếp theo mà chương trình có thể cần dựa trên luồng này và nạp trước chúng vào bộ nhớ cache của bộ xử lý. Các phương pháp Hardware Prefetching: *Stride Prefetcher*, *Stream Buffer*, *Irregular Temporal Prefetching*
2. **Software Prefetching:** Chèn các lệnh gợi ý vào mã nguồn. Các lệnh tìm nạp trước ảnh hưởng đến hiệu năng nhưng không ảnh hưởng đến logic chương trình. Việc tìm nạp trước bằng phần mềm có tác động tích cực đến hiệu năng khi được thực hiện đúng thời điểm trước khi tải thực tế. Nếu được thực hiện quá gần thời điểm truy cập, dữ liệu sẽ không được sao chép vào bộ nhớ cache; nếu được thực hiện quá xa, dữ liệu sẽ bị loại bỏ khi thực hiện tải

4.3 Lợi ích và Rủi ro

Lợi ích: Giảm thời gian chờ của CPU, giảm cache miss. Các chỉ số đánh giá Prefetching được kể đến: Coverage, Accuracy, Timeliness, Lateness, Pollution **Rủi ro:** *Cache Pollution* (làm bẩn cache) nếu dự đoán sai tồn bằng thông bộ nhớ, Prefetch sớm dữ liệu bị đẩy ra khỏi cache trước khi dùng, ngược lại Prefetch muộn sẽ không giảm được độ trễ.

4.4 Ứng dụng trong Web Caching và CDN(Content Delivery Network)

Các trình duyệt và CDN tối ưu hoá việc sử dụng tài nguyên, nâng cao hiệu suất bằng cách giảm độ trễ và tăng tốc độ phân phối nội dung, giúp trải nghiệm người dùng tốt hơn.

5 Phần 4: [Tên chủ đề phần 4]

(Nội dung phần này hiện đang để trống)

6 Kết luận

Tối ưu hóa cache là một quá trình đa tầng. Việc kết hợp linh hoạt các phương pháp giúp hệ thống hiện đại đạt được hiệu năng tối ưu, đáp ứng nhu cầu của các ứng dụng phức tạp.

Tài liệu

- [1] Anirudh Badam and Vivek S Pai. Ssdalloc: Hybrid ssd/ram memory management made easy. In *NSDI*, volume 11, pages 1–1, 2011.